



MONASH University

FIT3003 - A2

Business Intelligence and Data Warehousing

Beh Zhi Yin 33561656

Table of contents

Data Cleaning.....	3
Star schema diagrams for design task A.....	23
Design Task B - Suggestions on Increasing the Granularity of Fact Tables.....	23
Slowly Changing Dimensions (SCD) and Temporal Dimension.....	26
Table Structure (Design Task A).....	29
Descriptive Analysis.....	36
Finding 1 - Seasonal Trend in Total Revenue (Hire + Sales).....	36
Finding 2 - Branch Performance by Total Revenue (2018-2020).....	39
Finding 3 - Category Revenue Trend and Transaction Preference (2018-2020).....	42
Finding 4 - Customer Type Behaviour (2018-2020).....	46
Finding 5 - Sales Scale Distribution (High-Value vs Medium-Value, 2018-2020).....	47
Appendix.....	51
Data Cleaning Strategies.....	51
Reason Details - record data at the transaction level.....	52
References.....	54

Data Cleaning

Using these [data cleaning strategies](#), I identified seven data quality issues in the dataset.

Error 1: Table Category (Null Value Problems)

I find NULL value for category_id 15. Because category_id 15 also appears in the equipment table, it should not be deleted, as it does not affect the overall data count. Therefore, I only update the null values to 'Others' so that it will not cause any issues in future calculations.

Before:

```
Select * From CATEGORY;
```

All rows fetched: 15 in 0.148 seconds

	CATEGORY_ID	CATEGORY_DESCRIPTION
1	1	Access
2	2	Air Compressor
3	3	Compaction
4	4	Concrete
5	5	Earthmoving
6	6	Generators
7	7	Landscaping
8	8	Lighting
9	9	Plumbing
10	10	Rail
11	11	Safety
12	12	Site Equipment
13	13	Trailers
14	14	Vehicles
15	15	null

```
SELECT *  
  
FROM EQUIPMENT
```

```
WHERE CATEGORY_ID = 15;
```

All rows fetched: 158 in 0.610 seconds

	EQUIPMENT_ID	EQUIPMENT_NAME	EQUIPMENT_PRICE	MANUFACTURE_YEAR	MANUFACTURER	CATEGORY_ID
156	156	TIPPER 5T	31200	2013	JCB	14
157	157	TRAILER - LIVESTOCK	14400	2010	Volvo	14
158	158	EXCAVATOR - POST HOLE ATTACHMENT SUIT 3.5T	12200	2017	HITACHI	15

After:

```
UPDATE category  
  
SET category_description = 'Others'  
  
WHERE category_id = 15;
```

All rows fetched: 15 in 0.186 seconds

CATEGORY_ID	CATEGORY_DESCRIPTION
8	Lighting
9	Plumbing
10	Rail
11	Safety
12	Site equipment
13	Trailers
14	Vehicles
15	Others

Error 2: Table Customer (Duplication problems)

I checked which customer_id values appeared more than once and were not unique, and I found that CUSTOMER_ID = 52 had 4 duplicate rows. I decided to delete the duplicates to maintain the cleanliness and efficiency of the database and to ensure that future calculations remain accurate, as all primary keys in the customer table should be unique and not repeated.

Before:

```
SELECT CUSTOMER_ID, COUNT(*) AS duplicate_customer_id  
  
FROM CUSTOMER  
  
GROUP BY CUSTOMER_ID  
  
HAVING COUNT(*) > 1;
```

137

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPUT SQL HISTORY TASK MONITOR

All rows fetched: 1 in 0.129 seconds

	CUSTOMER_ID	DUPLICATE_CUSTOMER_ID
1	52	4

CUSTOMER_ID 52 is indeed repeated four times in the result.

```
SELECT * FROM CUSTOMER WHERE CUSTOMER_ID = 52;
```

All rows fetched: 4 in 0.132 seconds

	CUSTOMER_ID	CUSTOMER_TYPE_ID	NAME	GENDER	ADDRESS_ID	PHONE	EMAIL
1	52	2	Abbie Maddie	Male	52	904 627 9038	amaddie1@columbia.edu
2	52	2	Abbie Maddie	Male	52	904 627 9038	amaddie1@columbia.edu
3	52	2	Abbie Maddie	Male	52	904 627 9038	amaddie1@columbia.edu
4	52	2	Abbie Maddie	Male	52	904 627 9038	amaddie1@columbia.edu

After:

```
-- keep customer_id 52 for one row only, the rest
three delete

DELETE FROM CUSTOMER a

WHERE a.CUSTOMER_ID = 52

AND ROWID NOT IN (

    SELECT MIN(ROWID)

    FROM CUSTOMER b

    WHERE b.CUSTOMER_ID = 52

);
```

I filtered CUSTOMER_ID = 52 to ensure that only one row remains in the customer table, keeping the record unique and consistent with the primary key requirement.

```
SELECT * FROM CUSTOMER WHERE CUSTOMER_ID = 52;
```

All rows fetched: 1 in 0.116 seconds

	CUSTOMER_ID	CUSTOMER_TYPE_ID	NAME	GENDER	ADDRESS_ID	PHONE	EMAIL
1	52	2	Abbie Maddie	Male	52	904 627 9038	amaddie1f@columbia.edu

Error 3: Table Customer (Inconsistent values)

There is a mismatch between the gender values in the gender column of the CUSTOMER table and the gender column of the STAFF table. I updated the CUSTOMER table so that the gender values match the format used in the STAFF table ('Female' and 'Male', instead of 'F' and 'M').

Even though this issue does not significantly affect the current database, I made the correction to ensure the database is consistent and standardized, which helps reduce potential miscalculations in future tasks.

Before:

```
SELECT DISTINCT GENDER FROM CUSTOMER;
```

All rows fetched: 4 in 0.127 seconds

	GENDER
1	Male
2	M
3	Female
4	F

```
SELECT DISTINCT GENDER FROM STAFF;
```

rows fetched: 2 in 0.124 seconds

	GENDER
1	Male
2	Female

After:

The CUSTOMER table's gender column is now standardized to include only 'Female' or 'Male', matching the format used in the STAFF table for consistency.

```
-- Gender check

SELECT DISTINCT GENDER FROM CUSTOMER;

UPDATE CUSTOMER

SET GENDER = 'Male'

WHERE UPPER(GENDER) = 'M';

UPDATE CUSTOMER

SET GENDER = 'Female'

WHERE UPPER(GENDER) = 'F';
```

rows fetched: 2 in 0.129 seconds

	GENDER
1	Male
2	Female

Error 4: Table Equipment (Inconsistent values)

I found that 'HITACHI' and 'Hitachi' refer to the same manufacturer but were written in different formats - one in all capital letters ('HITACHI')

and the other with only the first letter capitalized ('Hitachi'). I updated the values in the manufacturer column of the EQUIPMENT table to ensure consistent capitalization, following the same format used for other manufacturer names in the column.

Before:

```
--check manufacturer name

SELECT DISTINCT manufacturer FROM equipment;

--found Hitachi and HITACHI for the manufacturer name
```

13 rows fetched: 13 in 0.142 seconds

	MANUFACTURER
1	Hitachi
2	Volvo
3	Kobelco
4	Komatsu
5	Case
6	Bobcat
7	HITACHI
8	JCB
9	Kenworth
10	Yanmar
11	Caterpillar
12	Isuzu
13	Kubota

After:

```
select * from equipment where manufacturer = 'Hitachi';

UPDATE EQUIPMENT

SET MANUFACTURER = 'Hitachi'

WHERE MANUFACTURER = 'HITACHI';
```

ows fetched: 12 in 0.131 seconds

	MANUFACTURER
1	Hitachi
2	Volvo
3	Kobelco
4	Komatsu
5	Case
6	Bobcat
7	JCB
8	Kenworth
9	Yanmar
10	Caterpillar
11	Isuzu
12	Kubota

```
select * from equipment where manufacturer = 'Hitachi';
```

217 select * from equipment where manufacturer = 'HITACHI';

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPUT SQL HISTORY TASK MONITOR

All rows fetched: 1 in 0.130 seconds

	EQUIPMENT_ID	EQUIPMENT_NAME	EQUIPMENT_PRICE	MANUFACTURE_YEAR	MANUFACTURER	CATEGORY_ID
1	158	EXCAVATOR - POST HOLE ATTACHMENT SUIT 3.5T	12200	2017	HITACHI	15

Error 5: Table CustomerType (Inconsistent values)

I found an invalid description inconsistency where the value 'Business' appeared in two different forms ('Business' and 'business'). I corrected this issue by deleting the duplicate 'business' row, which also repeated the CustomerTypeID = 2 value, to ensure consistent formatting across the column.

Before:

```
--check categorytype table

select * from CUSTOMERTYPE;

-- repeated customer_type_id 2. with business capital letter
```

rows fetched: 3 in 0.179 seconds

	CUSTOMER_TYPE_ID		DESCRIPTION
1		1	Individual
2		2	Business
3		2	business

After :

```

UPDATE CUSTOMERTYPE

SET DESCRIPTION = 'Business'

WHERE DESCRIPTION = 'business';

DELETE FROM CUSTOMERTYPE

WHERE CUSTOMER_TYPE_ID = 2

AND ROWID NOT IN (

    SELECT MIN(ROWID)

    FROM CUSTOMERTYPE

    WHERE CUSTOMER_TYPE_ID = 2

);

select * from CUSTOMERTYPE;

```

All rows fetched: 2 in 0.149 seconds

	CUSTOMER_TYPE_ID		DESCRIPTION
1		1	Individual
2		2	Business

Error 6: Table Hire (Relationship problems or constraint violations & Miscalculation)

First, I found that the equipment_id, customer_id, and staff_id for hire_id = 301 are invalid.

Specifically, staff_id = 174 may belong to a staff member who has retired or left the company, while customer_id = 181 is not recorded in the CUSTOMER table - the last valid customer_id is 150, so 181 appears to be an incorrect or invalid entry. Additionally, equipment_id = 190 does not exist in the EQUIPMENT table, indicating that this record was likely entered incorrectly and needs correction or removal.

Before:

```
--check foreign key: equipment_id

SELECT *

FROM HIRE

WHERE EQUIPMENT_ID NOT IN (

    SELECT EQUIPMENT_ID FROM EQUIPMENT

);

-- equipment_id 190 is not in the equipment table
```

All rows fetched: 1 in 0.124 seconds

	HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE_PRICE	TOTAL_HIRE_PRICE	CUSTOMER_ID	STAFF_ID
1	301	08/12/2020 12:00:00	08/12/2020 12:00:00	190	1	300	300	181	174

As the last recorded equipment_id in the EQUIPMENT table is 158. This means 190 is an impossible value and was likely entered incorrectly.

```
SELECT * FROM EQUIPMENT WHERE EQUIPMENT_ID = 190;

select equipment_id from equipment;
```

Rows fetched: 158 in 0.599 seconds

EQUIPMENT_ID
156
157
158

The last recorded customer_id in the CUSTOMER table is 150, so having customer_id = 181 does not make sense.

```
SELECT * FROM CUSTOMER WHERE CUSTOMER_ID = 181;

SELECT CUSTOMER_ID FROM CUSTOMER ;
```

15 rows fetched: 150 in 0.426 seconds

	CUSTOMER_ID	
148		148
149		149
150		150

The last recorded staff_id in the STAFF table is 50, so having staff_id = 174 does not make sense, even if the staff member might have retired or left the company.

```
SELECT * FROM STAFF WHERE STAFF_ID = 174;

select staff_id from staff;
```

51 rows fetched: 50 in 0.391 seconds

	STAFF_ID	
48		48
49		49
50		50

Since all three foreign keys (EQUIPMENT_ID, CUSTOMER_ID, and STAFF_ID) are invalid and cannot be corrected, I decided to delete this row to maintain accuracy, and consistency within the database.

After:

```
DELETE FROM HIRE

WHERE HIRE_ID = 301;

SELECT *
```

```
FROM HIRE
```

```
WHERE HIRE_ID = 301;
```

MS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULT SCRIPT OUTPUT SQL HISTORY TASK MONITOR						
s fetched: 0 in 0.125 seconds						
HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE_PRICE	TOTAL_HIRE_PRICE

It still does not make sense because the staff_id values (223, 123, and 85) in these records do not exist in the STAFF table, and the values are too large to be considered valid since the last recorded staff_id is 50. Additionally, the record with hire_id = 303 has a start date in the year 2090, which is clearly incorrect.

The hire_id = 303 and 304 records also show negative total hire prices, which are obviously invalid. The hire_id = 302 record was also will be deleted because the total hire price calculation was wrong - it forgot to multiply by the number of hire days, and the staff_id = 123 is also invalid.

Since there was no additional information available to accurately correct these records, I decided to delete all invalid rows (hire_id 301-304) to ensure the data remains clean, accurate, and logically consistent, preventing miscalculations in future analysis.

Before:

```
--check foreign key: staff_id

SELECT *

FROM HIRE

WHERE STAFF_ID NOT IN (

    SELECT STAFF_ID FROM STAFF

);

--staff_id 223,123,85 is not in the staff table
```

All rows fetched: 3 in 0.143 seconds

	HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE_PRICE	TOTAL_HIRE_PRICE	CUSTOMER_ID	STAFF_ID
1	303	25/01/2020 12:00:00	27/12/2020 12:00:00	43	3	50	-150	53	223
2	302	05/12/2020 12:00:00	17/10/2020 12:00:00	21	2	100	200	111	123
3	304	08/12/2020 12:00:00	08/12/2020 12:00:00	114	1	350	-1	34	85

Because the last registered staff_id is 50, it does not make sense for the table to contain staff_id values of 85, 123, or 223. These values are too large to be valid, so it is impossible that these staff members simply left or retired. Instead, the data was likely entered incorrectly, meaning the records in this table should be considered invalid and corrected or deleted to maintain data accuracy.

```
--check foreign key: equipment_id

SELECT *

FROM HIRE

WHERE END_DATE < START_DATE;

--hire_id: 302, end date is earlier than start date
```

All rows fetched: 1 in 0.153 seconds

	HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE_PRICE	TOTAL_HIRE_PRICE	CUSTOMER_ID	STAFF_ID
1	302	05/12/2020 12:00:00	17/10/2020 12:00:00	21	2	100	200	111	123

The start date is later than the end date for hire_id = 302, so this row was entered incorrectly.

```
-- date is outside from April 2018 to December 2020

SELECT *

FROM HIRE

WHERE START_DATE < DATE '2018-04-01'

    OR START_DATE > DATE '2020-12-31'

    OR END_DATE < DATE '2018-04-01'

    OR END_DATE > DATE '2020-12-31';

--hire_id 303 start date and end date is not valid
```

All rows fetched: 1 in 0.107 seconds

	HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE_PRICE	TOTAL_HIRE_PRICE	CUSTOMER_ID	STAFF_ID
1	303	25/01/2090 12:00:00	27/12/2099 12:00:00	43	3	50	-150	53	223

According to the assignment specification,

required for the management's daily operation. The operational database records the transaction from April 2018 to December 2020. However, since the staff at MonEquip has limited database

All data should be recorded between April 2018 and December 2020. Therefore, the hire_id = 303 record must have been entered incorrectly, as its start date is in the year 2090, which is impossible since the current year is only 2025. In addition, the total hire price shows a negative value (-150), which further confirms that the data in this row is invalid and incorrectly recorded.

```
--total_hire_price is having negative value or 0

SELECT *
FROM HIRE
WHERE QUANTITY <= 0

      OR UNIT_HIRE_PRICE <= 0

      OR TOTAL_HIRE_PRICE <= 0;

--hire_id 303 & 304
```

All rows fetched: 2 in 0.150 seconds

HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE_PRICE	TOTAL_HIRE_PRICE	CUSTOMER_ID	STAFF_ID	
1	303	25/01/2090 12:00:00	27/12/2099 12:00:00	43	3	50	-150	53	223
2	304	08/12/2020 12:00:00	08/12/2020 12:00:00	114	1	350	-1	34	85

Hence, I decided to delete rows 302, 303, and 304 as well. Although the customer_id and equipment_id in hire_id = 304 are valid, the staff_id = 85 does not make sense, as it exceeds the last valid staff_id (50). Moreover, the total hire price is recorded as -1, which is incorrect - it should be 175, indicating a calculation error. Therefore, I assume the hire_id = 304 record was also entered incorrectly, and I deleted it to maintain data accuracy & consistency in the database.

After:

```
DELETE FROM HIRE WHERE HIRE_ID = 302;

DELETE FROM HIRE WHERE HIRE_ID = 303;

DELETE FROM HIRE WHERE HIRE_ID = 304;
```

```
select *
from hire
where hire_id = 301
or hire_id = 302
or hire_id = 303
or hire_id = 304;
```

rows fetched: 0 in 0.121 seconds

HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE_PRICE	TOTAL_HIRE_PRICE	CUSTOMER_ID	STAFF_ID
---------	------------	----------	--------------	----------	-----------------	------------------	-------------	----------

I found a calculation error in the total_hire_price column: 165 rows were calculated incorrectly.

```
--total_hire_price is calculated wrongly

--total 165 rows got problems.

SELECT *

FROM (

    SELECT HIRE_ID,

           START_DATE,

           END_DATE,

           QUANTITY,

           UNIT_HIRE_PRICE,

           TOTAL_HIRE_PRICE,

           CASE

               WHEN (TO_DATE(END_DATE, 'DD/MM/YY') -

TO_DATE(START_DATE, 'DD/MM/YY')) < 1

               THEN 0.5 * UNIT_HIRE_PRICE * QUANTITY
```

```

        ELSE (TO_DATE(END_DATE, 'DD/MM/YY') -
TO_DATE(START_DATE, 'DD/MM/YY')) * UNIT_HIRE_PRICE * QUANTITY

        END AS CALCULATED_TOTAL

FROM HIRE

)

WHERE TOTAL_HIRE_PRICE <> CALCULATED_TOTAL

ORDER BY HIRE_ID;

```

Fetched 165 rows in 0.704 seconds.

	HIRE_ID	START_DATE	END_DATE	QUANTITY	UNIT_HIRE_PRICE	TOTAL_HIRE_PRICE	CALCULATED_TOTAL	
1	1	11/05/2018 12:00:00	14/05/2018 12:00:00	3	80	240	720	
2	2	17/05/2018 12:00:00	20/05/2018 12:00:00	2	660	1320	3960	
3	4	21/05/2018 12:00:00	25/05/2018 12:00:00	1	540	540	2160	
4	5	21/05/2018 12:00:00	23/05/2018 12:00:00	2	600	1200	2400	
5	6	22/05/2018 12:00:00	26/05/2018 12:00:00	2	500	1000	4000	
6	7	24/05/2018 12:00:00	28/05/2018 12:00:00	3	140	420	1680	
7	8	25/05/2018 12:00:00	28/05/2018 12:00:00	3	80	240	720	
8	11	12/06/2018 12:00:00	14/06/2018 12:00:00	2	450	900	1800	
9	18	17/07/2018 12:00:00	14/07/2018 12:00:00	1	600	1000	1600	

I first updated these rows where the total hire price was incorrectly calculated because the system forgot to multiply by the number of days the customer hired the equipment.

To confirm the correction, I also wrote an update statement to check for same-day returns, and as expected, it showed 0 rows updated. This confirms that all 165 incorrect rows were due to missing the multiplication by the hire duration (end_date - start_date) in the calculation.

After:

```

UPDATE HIRE

SET TOTAL_HIRE_PRICE = 0.5 * UNIT_HIRE_PRICE * QUANTITY

WHERE (TO_DATE(END_DATE, 'DD/MM/YYYY HH24:MI:SS') - TO_DATE(START_DATE,
'DD/MM/YYYY HH24:MI:SS')) < 1

AND TOTAL_HIRE_PRICE <> 0.5 * UNIT_HIRE_PRICE * QUANTITY;

```

```
0 rows updated.
```

```
UPDATE HIRE
```

```
SET TOTAL_HIRE_PRICE =
```

```
    (TO_DATE(END_DATE, 'DD/MM/YYYY HH24:MI:SS') - TO_DATE(START_DATE,  
'DD/MM/YYYY HH24:MI:SS'))
```

```
    * UNIT_HIRE_PRICE * QUANTITY
```

```
WHERE (TO_DATE(END_DATE, 'DD/MM/YYYY HH24:MI:SS') - TO_DATE(START_DATE,  
'DD/MM/YYYY HH24:MI:SS')) >= 1
```

```
AND TOTAL_HIRE_PRICE <>
```

```
    (TO_DATE(END_DATE, 'DD/MM/YYYY HH24:MI:SS') - TO_DATE(START_DATE,  
'DD/MM/YYYY HH24:MI:SS'))
```

```
    * UNIT_HIRE_PRICE * QUANTITY;
```

```
165 rows updated.
```

Now, there are no rows with incorrect calculations in the total_hire_price column.

```
SELECT *
```

```
FROM (
```

```
    SELECT HIRE_ID,
```

```
           START_DATE,
```

```
           END_DATE,
```

```
           QUANTITY,
```

```
           UNIT_HIRE_PRICE,
```

```
           TOTAL_HIRE_PRICE,
```

```

CASE
    WHEN (TO_DATE(END_DATE, 'DD/MM/YY') -
TO_DATE(START_DATE, 'DD/MM/YY')) < 1

        THEN 0.5 * UNIT_HIRE_PRICE * QUANTITY

    ELSE (TO_DATE(END_DATE, 'DD/MM/YY') -
TO_DATE(START_DATE, 'DD/MM/YY')) * UNIT_HIRE_PRICE * QUANTITY

    END AS CALCULATED_TOTAL

FROM HIRE

)

WHERE TOTAL_HIRE_PRICE <> CALCULATED_TOTAL

ORDER BY HIRE_ID;

```

All rows fetched: 0 in 0.165 seconds

HIRE_ID	START_DATE	END_DATE	QUANTITY	UNIT_HIRE_PRICE	TOTAL_HIRE_PRICE	CALCULATED_TOTAL
---------	------------	----------	----------	-----------------	------------------	------------------

ERROR 7: Sales Table (Incorrect Value)

To check for invalid values such as negative or zero quantities, I used two SQL statements - one to detect negative and zero values in the quantity, unit sales price, or total sales price columns, and another to verify whether the total sales price was calculated correctly.

```

--check invalid value such as negative and 0 (to be analyse)

SELECT *

FROM SALES

WHERE QUANTITY <= 0

```

```
OR UNIT_SALES_PRICE <= 0

OR TOTAL_SALES_PRICE <= 0;
```

```
SELECT SALES_ID,

       SALES_DATE,

       EQUIPMENT_ID,

       QUANTITY,

       UNIT_SALES_PRICE,

       TOTAL_SALES_PRICE,

       (UNIT_SALES_PRICE * QUANTITY) AS CALCULATED_TOTAL

FROM SALES

WHERE TOTAL_SALES_PRICE <> (UNIT_SALES_PRICE * QUANTITY)

ORDER BY SALES_ID;
```

All rows fetched: 1 in 0.205 seconds

	SALES_ID	SALES_DATE	EQUIPMENT_ID	QUANTITY	UNIT_SALES_PRICE	TOTAL_SALES_PRICE	CALCULATED_TOTAL	
1	151	15/12/2020 12:00:00		20	-3	45500	182000	-136500

From the results, sales_id = 151 was the only row with an incorrect calculation. I first attempted to update the quantity value to positive 3, but the calculation error of the total_sales_price persisted, which indicated that the issue was not caused by the negative quantity value (-3).

After:

```
UPDATE SALES

SET QUANTITY = 3

WHERE SALES_ID = 151

AND QUANTITY = -3;
```

```

SELECT SALES_ID,

       SALES_DATE,

       EQUIPMENT_ID,

       QUANTITY,

       UNIT_SALES_PRICE,

       TOTAL_SALES_PRICE,

       (UNIT_SALES_PRICE * QUANTITY) AS CALCULATED_TOTAL

FROM SALES

WHERE TOTAL_SALES_PRICE <> (UNIT_SALES_PRICE * QUANTITY)

ORDER BY SALES_ID;

```

All rows fetched: 1 in 0.149 seconds

SALES_ID	SALES_DATE	EQUIPMENT_ID	QUANTITY	UNIT_SALES_PRICE	TOTAL_SALES_PRICE	CALCULATED_TOTAL
1	151 15/12/2020 12:00:00		20	3	45500	182000

By dividing the total sales price (182000) by the unit sales price (45500), I obtained a result of 4, meaning the quantity should be 4 instead of -3 or 3. I then updated the quantity value to 4 and rechecked the total sales price calculation.

After:

```

UPDATE SALES

SET QUANTITY = 4

WHERE SALES_ID = 151

AND QUANTITY = 3;

```

The validation query confirmed that there were no further miscalculation errors, indicating that the data cleaning process has been successfully completed.

```

SELECT SALES_ID,

```

```
    SALES_DATE,

    EQUIPMENT_ID,

    QUANTITY,

    UNIT_SALES_PRICE,

    TOTAL_SALES_PRICE,

    (UNIT_SALES_PRICE * QUANTITY) AS CALCULATED_TOTAL

FROM SALES

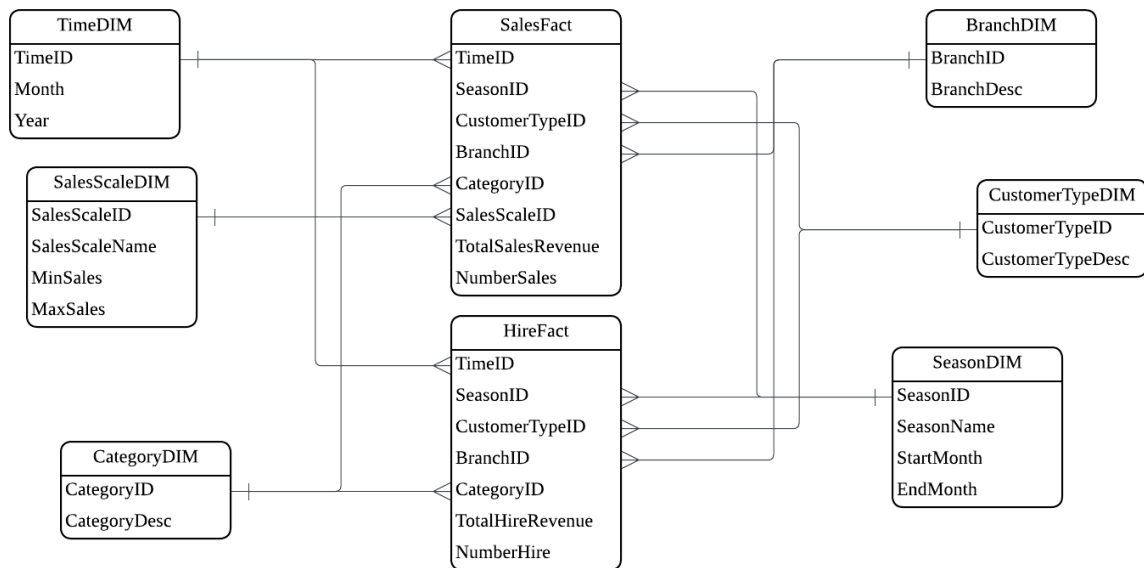
WHERE TOTAL_SALES_PRICE <> (UNIT_SALES_PRICE * QUANTITY)

ORDER BY SALES_ID;
```

All rows fetched: 0 in 0.155 seconds

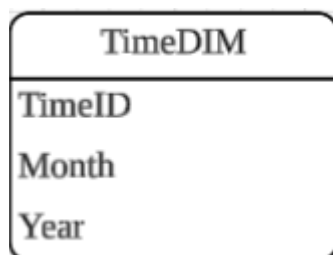
SALES_ID	SALES_DATE	EQUIPMENT_ID	QUANTITY	UNIT_SALES_PRICE	TOTAL_SALES_PRICE	CALCULATED_TOTAL
----------	------------	--------------	----------	------------------	-------------------	------------------

Star schema diagrams for design task A



Design Task B - Suggestions on Increasing the Granularity of Fact Tables

Task A star schema summarises Sales and Hire at month/year level. To enable drill-down analysis (daily peaks, item performance), we lower the level of aggregation and define a consistent, finer grain for each fact.



1. Increase the time detail to a daily level.

Replace Month/Year grain with TimeID = YYYYMMDD.

Benefit:

- Analyse day-to-day variations, holiday effects, or short-term campaigns.

2. Record data at the transaction level. [\[Reason Details\]](#)

Include SalesID and HireID in the SalesFact and HireFact tables so that each record represents a single sales or hire transaction instead of an aggregated total.

This change helps avoid combining multiple transactions into one record, such as cases where NumberHire = 2 due to two transactions being aggregated together.

Benefit:

- Enables detailed transaction-level analysis, such as identifying large individual sales or repeated customer hires.
- Supports drill-down from monthly summaries to individual transactions.
- Improves accuracy in evaluating staff performance, customer buying behaviour, and equipment utilisation across branches.

3. Create Equipment Dimension.

Attributes: EquipmentID, EquipmentName, EquipmentPrice, Manufacturer, ManufacturerYear

Benefit:

- Helps the company see which equipment generates the highest sales and hire revenue.
- Identifies the most and least popular products.
- Makes it easier to compare performance between different manufacturers.

4. Store unit price and quantity separately.

Include columns such as Quantity, UnitSalesPrice, and UnitHirePrice in

the fact tables so that each record shows the actual unit price and quantity at the time of the transaction.

TotalSalesRevenue and TotalHireRevenue can then be derived as $\text{Quantity} \times \text{UnitPrice}$.

Adjustment:

Record only the actual price used in each transaction instead of adding “PreviousPrice” or “CurrentPrice” columns.

Each fact record will already represent the correct historical price for that specific sale or hire.

If price-tracking over time is required in the future, a Slowly Changing Dimension (Type 2) can be implemented in EquipmentDim to capture effective date ranges for price updates.

Benefit:

- Ensures historical pricing accuracy even when prices fluctuate.
- Avoids redundant columns and simplifies maintenance.
- Supports calculation of revenue, profit, and pricing trend analysis at the transaction level.

5. Add a Staff Dimension

Create a StaffDim table to store staff-related details such as:

StaffID, FirstName, LastName, Gender, Phone, Email

Each staff member will be linked to the fact tables through StaffID, allowing analysis of how much sales or hire revenue each employee contributes.

Benefit:

- Enables evaluation of staff productivity, revenue contribution, and transaction handling efficiency.

6. Replace CustomerTypeDim with CustomerDim

Attributes: CustomerID, CustomerType, Name, Gender, Phone, Email

Benefit:

- Enables more detailed customer information and allows immediate analysis based on attributes such as gender.

Slowly Changing Dimensions (SCD) and Temporal Dimension

A *Slowly Changing Dimension (SCD)* defines how changes in dimension attributes are tracked and stored over time. The five main SCD types manage historical data differently:

SCD Type	Description	Purpose / Example
Type 1 - Overwrite	Replaces old values with the latest value of the records, without recording the history of changes in the dimension	Used for small corrections, such as hire price updated
Type 2 - Add New Record	Inserts a new row when an attribute changes, with <i>StartDate</i> , <i>EndDate</i> , and <i>CurrentFlag</i> . Preserves full history.	Suitable for tracking price changes over time or staff role updates.

Type 3 - Add New Column	Stores both current and previous values in the same record for limited comparison.	Useful for short-term tracking, such as comparing last season's price to current.
Type 4 - Separate History Table	Keeps the current records in the main dimension table, while historical changes are stored in a separate history (temporal) table. The main dimension excludes the changing attribute (e.g., UnitHirePrice), and the history table includes fields such as UnitHirePrice, StartDate, and EndDate.	Ideal when attribute changes occur frequently and historical data needs to be accessed separately.
Type 6 - (Combination of Type 2 and Type 3)	• Combines Type 2 (add new record) and Type 3 (add new column). • Keeps full history while storing "current" and "previous" values for comparison. • No separate identifier needed for the same entity. • Cardinality changes from 1-M to M-M. • Solutions: (1) add surrogate key	Useful when both historical tracking and quick comparison are required. In MonEquip, it can record hire price changes over time (Type 2) and store "current" and "previous" price columns (Type 3).

	(2) add Start/End Date to Fact (3) use a bridge table.	
--	---	--

In MonEquip, the only attribute that varies over time is the unit hire price. Therefore, a temporal dimension called EquipmentPriceDim should be implemented to record historical hire price changes for each equipment item. Each record includes EquipmentID, UnitHirePrice, StartDate, EndDate, and CurrentFlag, allowing the system to identify the active price version. This enables the HireFact table to link to the correct price valid at the time of each hire transaction.

For Design Task B, which increases granularity to the transaction level, SCD Type 2 is preferred as it records each price change with effective-date versioning and ensures every hire transaction reflects the accurate price at that time.

Table Structure (Design Task A)

Time Dimension

All rows fetched: 33 in 0.164 seconds

TIMEID	MONTH	YEAR
1	201804	4
2	201910	10
3	201805	5
4	201811	11
5	201812	12
6	201908	8
7	202006	6
8	202011	11
9	201903	3
10	201911	11
11	201810	10
12	201901	1
13	201904	4
14	201909	9
15	202002	2
16	202005	5
17	201905	5
18	201906	6
19	201912	12
20	202008	8
21	202009	9
22	202010	10
23	201808	8
24	202001	1
25	202007	7
26	201806	6
27	201807	7
28	201809	9
29	201902	2
30	201907	7
31	202001	1
32	202004	4
33	202012	12

Season Dimension

All rows fetched: 4 in 0.147 seconds

SEASONID	SEASONNAME	STARTMONTH	ENDMONTH
1	911 Spring	9	11
2	1202 Summer	12	2
3	305 Autumn	3	5
4	606 Winter	6	8

Customer Type Dimension

All rows fetched: 2 in 0.136 seconds

CUSTOMERTYPEID	CUSTOMERTYPEDESC
1	Individual
2	Business

Branch Dimension

All rows fetched: 15 in 0.163 seconds

BRANCHID	BRANCHDESC
1	1 Caulfield
2	2 Chadstone
3	3 Cheltenham
4	4 Clayton
5	5 Dandenong
6	6 Docklands
7	7 Eltham
8	8 Fitzroy
9	9 Geelong
10	10 Hughesdale
11	11 Pakenham
12	12 Parkville
13	13 Prahran
14	14 Richmond
15	15 Toorak

Category Dimension

All rows fetched: 15 in 0.149 seconds

	CATEGORYID		CATEGORYDESC
1		1	Access
2		2	Air Compressor
3		3	Compaction
4		4	Concrete
5		5	Earthmoving
6		6	Generators
7		7	Landscaping
8		8	Lighting
9		9	Plumbing
10		10	Rail
11		11	Safety
12		12	Site Equipment
13		13	Trailers
14		14	Vehicles
15		15	Others

Sales Scale Dimension

All rows fetched: 3 in 0.157 seconds

SALESCALEID	SALESCALENAME	MINSALES	MAXSALES
1	Low		4999.99
2	Medium	5000	10000
3	High	10000.01	99999999

I assigned the value 999999999 because if I left the max sales value as NULL for high sales, it might cause confusion for others. They could think it was forgotten to be updated or that an error occurred.

Sales Fact

All rows fetched: 150 in 0.096 seconds

	TIMEID	SEASONID	CUSTOMERTYPEID	BRANCHID	CATEGORYID	SALESSCALEID	NUMBERSALES	TOTALSALESREVENUE	
1		202009	911	1	9	1	3	1	94500
2		202006	608	1	4	5	3	1	240000
3		201812	1202	2	6	6	3	1	68000
4		202005	305	2	4	7	3	1	117000
5		201810	911	2	4	8	3	1	68000
6		201811	911	2	5	10	3	1	96000
7		202009	911	1	7	11	3	1	52000
8		201807	608	1	8	11	3	1	11200
9		201809	911	2	4	11	3	1	38400
10		201804	305	2	15	11	3	1	18000
11		201808	608	2	1	13	3	1	14000
12		202004	305	2	6	14	3	1	32000
13		201912	1202	1	10	14	3	1	72000
14		201904	305	1	15	1	3	1	156000
15		201905	305	1	14	2	3	1	29400
16		202003	305	1	15	3	3	1	60000
17		201807	608	2	12	3	3	1	60000
18		201808	608	1	12	3	3	1	65000
19		201808	608	2	4	3	3	1	72000
20		202003	305	2	9	4	3	1	22500
21		201807	608	2	4	5	3	1	338000
22		202005	305	2	12	5	3	1	54000
23		201907	608	1	4	6	3	1	91200
24		202005	305	1	12	7	3	1	48000
25		202009	911	2	12	7	3	1	117000
26		201807	608	2	4	9	3	1	19200
27		201808	608	2	1	9	2	1	5600
28		202006	608	2	1	9	3	1	138600
29		201808	608	1	4	11	3	1	43200
30		201911	911	1	9	13	3	1	28500
31		202008	608	2	2	13	3	1	48000

	TIMEID	SEASONID	CUSTOMERTYPEID	BRANCHID	CATEGORYID	SALESSCALEID	NUMBERSALES	TOTALSALESREVENUE	
32		201910	911	1	15	13	3	1	17600
33		201905	305	2	2	14	3	1	312000
34		201810	911	1	15	1	3	1	63000
35		201910	911	2	4	1	3	1	1540950
36		202010	911	2	6	2	3	1	198000
37		201807	608	2	2	3	3	1	260000
38		201906	608	2	15	3	3	1	72000
39		202002	1202	1	8	4	3	1	120000
40		202005	305	1	4	4	3	1	24000
41		202002	1202	1	6	4	3	1	89600
42		201807	608	1	13	5	3	1	162000
43		201909	911	2	12	6	3	1	120000
44		201804	305	2	4	9	3	1	33000
45		201912	1202	2	12	9	3	1	35200
46		202005	305	2	13	9	2	1	6400
47		201808	608	2	15	10	3	1	14400
48		201902	1202	1	13	10	3	1	144000
49		201905	305	1	6	12	3	1	86400
50		202005	305	1	13	12	3	1	21600
51		201905	305	2	5	14	3	1	18000
52		202010	911	1	4	14	3	1	72000
53		201908	608	1	11	2	3	1	90000
54		201908	608	2	4	3	3	1	18000
55		202011	911	1	14	6	3	1	78000
56		202011	911	1	4	6	3	1	112000
57		201903	305	1	11	6	3	1	60000
58		201905	305	2	11	7	3	1	72000
59		202012	1202	2	4	7	3	1	166400
60		201812	1202	1	13	8	3	1	99000
61		201810	911	1	15	9	3	1	43200
62		201809	911	2	7	9	3	1	11200
63		202012	1202	2	4	12	3	1	176000

64		201809	911	2	8	13	3	1	120000
65		201911	911	2	11	2	3	1	21600
66		202006	608	2	2	2	3	1	91000
67		202007	608	2	8	4	3	1	67500
68		202009	911	1	3	5	3	1	162000
69		201807	608	2	3	5	3	1	35000
70		201807	608	2	11	5	3	1	216000
71		201902	1202	2	12	6	3	1	120000
72		201809	911	2	4	6	3	1	99000
73		201807	608	1	9	7	3	1	63000
74		201811	911	1	4	7	3	1	126000
75		201911	911	2	14	7	3	1	112000
76		201901	1202	2	3	9	3	1	46800
77		201903	305	2	9	10	3	1	80000
78		201907	608	1	8	10	3	1	150000
79		201903	305	2	10	12	2	1	5000
80		201908	608	1	4	14	3	1	72000
81		201903	305	2	4	2	3	1	150000
82		202001	1202	1	13	3	3	1	65000
83		201812	1202	2	12	4	3	1	130000
84		201902	1202	1	10	5	3	1	48000
85		201906	608	1	15	6	3	1	45000
86		202009	911	1	11	6	3	1	148500
87		202009	911	2	6	7	3	1	32000
88		202011	911	1	6	7	3	1	39000
89		201912	1202	2	4	7	3	1	60000
90		202005	305	1	10	8	3	1	33600
91		201809	911	2	5	8	3	1	67200
92		202008	608	2	8	8	3	1	20000
93		201910	911	2	2	9	3	1	16800
94		202006	608	1	12	10	3	1	210000

95	202008	608	1	12	11	3	1	39000
96	201902	1202	1	15	11	3	1	16800
97	201901	1202	2	6	11	3	1	43200
98	201910	911	1	1	11	2	1	9600
99	202011	911	1	6	11	3	1	12600
100	201911	911	2	10	13	3	1	43200
101	202011	911	2	4	13	3	1	18000
102	202001	1202	2	1	14	3	1	140400
103	201809	911	2	6	1	3	1	214500
104	201909	911	1	4	1	3	1	88800
105	202005	305	1	1	2	3	1	130000
106	202012	1202	2	11	2	3	2	364000
107	202009	911	1	11	4	3	1	40000
108	202006	608	1	12	5	3	1	253500
109	202011	911	1	4	5	3	1	84500
110	202004	305	1	2	5	3	1	290400
111	202010	911	1	3	5	3	1	105600
112	202011	911	2	1	7	3	1	16800
113	202010	911	2	15	7	3	1	252000
114	202001	1202	2	7	7	3	1	11200
115	201902	1202	1	6	8	3	1	28000
116	202005	305	2	3	8	3	1	20000
117	202002	1202	2	6	9	3	1	11200
118	201903	305	2	4	10	3	1	19200
119	201810	911	1	6	10	3	1	58500
120	201901	1202	2	8	10	3	1	46800
121	201812	1202	2	12	11	3	1	36000
122	201907	608	1	1	11	3	1	18000
123	202003	305	2	13	11	3	1	52000
124	201905	305	1	4	11	3	1	28800
125	202007	608	1	4	13	3	1	42000

126	201809	911	1	4	13	3	1	50400
127	201911	911	2	2	2	3	1	91000
128	202007	608	2	12	2	3	1	132000
129	201808	608	2	15	3	3	1	18000
130	201907	608	2	4	3	3	1	52800
131	201909	911	2	1	3	3	1	18000
132	201912	1202	2	13	3	3	1	97200
133	201904	305	1	10	4	2	1	10000
134	201908	608	1	15	5	3	1	48000
135	201809	911	2	4	7	3	1	19200
136	201910	911	2	14	7	3	1	32000
137	201805	305	1	13	7	3	1	83200
138	201809	911	2	15	7	3	1	124800
139	202001	1202	2	3	7	3	1	44800
140	202012	1202	1	10	8	3	1	192000
141	201807	608	1	7	9	3	1	26400
142	201910	911	1	12	9	3	1	26400
143	201807	608	2	11	10	3	1	80000
144	202004	305	2	8	10	3	1	15600
145	202001	1202	2	14	11	3	1	95200
146	201901	1202	1	5	12	3	1	41600
147	202010	911	2	4	13	3	1	14400
148	201810	911	2	4	13	3	1	14000
149	201806	608	2	4	13	3	1	18000
150	202003	305	2	10	14	3	1	32000

Hire Fact

All rows fetched: 298 in 0.797 seconds

	TIMEID	SEASONID	CUSTOMERTYPEID	BRANCHID	CATEGORYID	NUMBERHIRE	TOTALHIREREVENUE
1	201807	608	1	4	2	1	3600
2	201807	608	1	6	13	1	240
3	201807	608	2	13	2	1	5400
4	201808	608	1	1	6	1	2100
5	201810	911	1	15	11	1	1400
6	201811	911	2	6	6	1	2160
7	201811	911	1	4	8	1	1800
8	201812	1202	2	6	8	1	1608
9	201812	1202	2	2	11	1	300
10	201812	1202	2	1	9	1	48
11	201812	1202	2	2	10	1	1560
12	201812	1202	2	13	5	1	900
13	201902	1202	2	11	4	1	200
14	201904	305	2	12	8	1	460
15	201905	305	1	1	6	1	1140
16	201906	608	2	2	7	1	3600
17	201906	608	1	11	8	1	320
18	201906	608	1	13	5	1	750
19	201908	608	1	4	6	1	420
20	201909	911	1	8	12	1	180
21	201909	911	2	4	2	1	900
22	201910	911	1	8	6	1	800
23	201911	911	1	13	12	1	960
24	201911	911	2	7	9	1	160
25	201912	1202	2	10	9	1	180
26	202001	1202	2	4	12	1	180
27	202002	1202	2	8	12	1	240
28	202007	608	1	1	3	1	1000
29	202008	608	1	15	3	1	800
30	202008	608	2	12	5	1	2800
31	202008	608	2	13	11	1	900
32	202008	608	1	6	11	1	90
33	202009	911	2	11	8	1	4440
34	202010	911	1	13	2	1	488
35	202010	911	1	11	5	1	100
36	202010	911	1	7	14	1	100
37	202011	911	1	10	4	1	100
38	201808	608	2	9	10	1	680
39	201902	1202	2	1	2	1	140
40	201806	608	1	15	6	1	1800
41	201807	608	2	10	13	1	440
42	201808	608	2	4	11	1	330
43	201809	911	1	5	9	1	80
44	201811	911	1	1	1	1	1200
45	201811	911	1	5	8	1	660
46	201812	1202	1	11	12	1	10
47	201901	1202	1	4	11	1	270
48	201903	305	1	4	5	1	330
49	201903	305	2	11	3	1	250
50	201904	305	1	4	1	1	1600
51	201905	305	1	10	2	1	540
52	201906	608	1	15	2	1	80
53	201906	608	2	12	13	1	1800
54	201907	608	2	15	8	1	140
55	201907	608	1	11	7	1	400
56	201908	608	2	10	13	1	200
57	201909	911	2	1	12	1	1260
58	201909	911	2	1	7	1	320
59	201911	911	2	10	12	1	720
60	202006	608	1	11	10	1	60
61	202007	608	1	4	10	1	1500
62	202008	608	1	1	7	1	1400
63	202008	608	1	4	14	1	2160
64	202010	911	2	1	10	1	1200
65	202011	911	1	11	10	1	2700
66	202011	911	2	13	7	1	450
67	202012	1202	1	15	11	1	620
68	201805	305	2	7	12	1	720
69	201810	911	2	8	1	1	2520
70	201805	305	1	10	12	1	720
71	201806	608	2	6	1	1	360
72	201808	608	1	8	3	1	360
73	201808	608	1	15	14	1	3200
74	201809	911	2	8	4	1	100
75	201809	911	1	4	11	1	400
76	201811	911	1	3	14	1	4800
77	201812	1202	1	11	1	1	890
78	201812	1202	1	11	8	1	3130
79	201903	305	1	12	9	1	80
80	201903	305	2	6	3	1	800
81	201905	305	1	5	13	1	1600
82	201905	305	1	5	5	1	1350
83	201905	305	2	13	7	1	225
84	201906	608	2	7	8	1	600
85	201906	608	2	14	13	1	800
86	201906	608	2	4	2	1	760
87	201910	911	1	2	5	1	7920
88	201911	911	2	2	9	1	480
89	201911	911	2	4	12	1	120
90	201912	1202	1	7	13	1	330
91	202001	1202	1	1	9	1	520
92	202002	1202	1	15	5	1	400
93	202002	1202	1	6	8	1	810
94	202003	305	2	14	8	1	105
95	202004	305	1	1	13	1	540
96	202006	608	2	8	12	1	50
97	202006	608	2	4	9	1	1080
98	202007	608	1	4	12	1	1680
99	202007	608	1	4	11	1	420

100	202009	911	1	12	2	1	675
101	202010	911	1	11	10	1	225
102	202010	911	1	10	9	1	80
103	201907	608	2	10	10	1	300
104	201901	1202	2	11	8	1	700
105	201905	305	1	1	7	1	1680
106	201907	608	1	8	13	1	200
107	201907	608	1	14	1	1	395
108	201908	608	1	12	13	1	600
109	201908	608	2	2	4	1	960
110	201908	608	2	1	12	1	1680
111	201909	911	1	6	6	1	500
112	201810	911	1	7	8	1	1320
113	201810	911	2	6	8	1	720
114	201810	911	2	4	9	1	5040
115	201811	911	1	10	2	1	3200
116	201811	911	1	4	13	1	1000
117	201812	1202	2	2	9	1	50
118	201812	1202	1	12	12	1	60
119	201902	1202	2	11	12	1	640
120	201904	305	2	13	2	1	3000
121	201905	305	2	10	4	1	250
122	201905	305	1	10	3	1	1440
123	201905	305	2	12	13	1	480
124	201906	608	1	9	10	1	4500
125	201908	608	2	1	3	1	600
126	201908	608	2	4	7	1	140
127	201910	911	1	15	13	1	120
128	201910	911	1	1	6	1	1350
129	201910	911	2	11	14	1	2460
130	202002	1202	2	8	13	1	200
131	202002	1202	1	11	6	1	2340
132	202002	1202	2	12	12	1	1800

133	202002	1202	2	4	12	1	240
134	202003	305	1	9	4	1	225
135	202004	305	2	10	5	1	360
136	202005	305	1	10	3	1	1800
137	202005	305	2	11	5	1	330
138	202005	305	2	1	13	1	400
139	202008	608	2	6	6	1	1680
140	202009	911	1	11	11	1	20
141	202009	911	2	10	1	1	1200
142	202009	911	1	5	1	1	405
143	202010	911	2	14	4	1	3000
144	202011	911	1	4	9	1	960
145	202011	911	2	15	1	1	540
146	201812	1202	2	10	6	1	500
147	201812	1202	2	15	12	1	210
148	201906	608	2	4	8	1	2040
149	201910	911	1	7	3	1	800
150	201005	305	1	14	5	1	4000
151	201005	305	2	4	11	1	170
152	201005	305	1	6	8	1	200
153	201808	608	1	2	6	1	110
154	201808	608	2	8	2	1	120
155	201809	911	2	13	1	1	1440
156	201810	911	2	13	4	1	160
157	201812	1202	1	2	11	1	280
158	201901	1202	2	7	5	1	750
159	201901	1202	2	8	1	1	900
160	201904	305	1	4	10	1	2640
161	201907	608	2	13	5	1	675
162	201907	608	2	15	12	1	100
163	201908	608	1	1	11	1	120
164	201908	608	1	11	8	1	660
165	201910	911	1	12	14	1	400
166	201910	911	2	15	9	1	880

167	201911	911	1	12	5	1	100
168	201911	911	2	8	6	1	600
169	201912	1202	1	10	8	1	560
170	201912	1202	1	11	3	1	1800
171	202001	1202	2	11	1	1	1110
172	202002	1202	1	15	1	1	1560
173	202003	305	1	15	7	1	1980
174	202003	305	2	9	11	1	100
175	202004	305	2	12	4	1	150
176	202004	305	1	15	6	1	2400
177	202005	305	2	6	13	1	780
178	202007	608	2	6	2	1	320
179	202009	911	2	8	5	1	4050
180	202009	911	1	13	3	1	1200
181	202010	911	1	7	7	1	2700
182	202011	911	2	10	10	1	500
183	201808	608	2	6	12	1	4800
184	201904	305	2	4	7	1	280
185	201909	911	2	10	1	1	1800
186	201909	911	1	3	3	1	360
187	201005	305	2	4	10	1	150
188	201007	608	2	1	8	1	500
189	201007	608	2	1	11	1	420
190	201008	608	1	10	5	1	1500
191	201008	608	2	9	13	1	100
192	201008	608	2	10	6	1	300
193	201008	608	1	14	10	1	600
194	201009	911	1	11	8	1	3840
195	201810	911	1	1	12	1	80
196	201810	911	2	12	9	1	1260
197	201901	1202	1	13	13	1	1440
198	201901	1202	2	6	5	1	1500

199	201901	1202	1	2	3	1	200
200	201905	305	2	4	6	1	450
201	201906	608	2	2	11	1	80
202	201907	608	1	9	10	1	500
203	201908	608	1	6	3	1	3240
204	201910	911	2	6	1	1	2100
205	201910	911	1	10	13	1	1800
206	201910	911	2	4	8	1	800
207	201912	1202	2	1	11	1	240
208	202002	1202	2	7	10	1	520
209	202003	305	2	6	2	1	1520
210	202003	305	1	12	12	1	240
211	202003	305	2	4	1	1	1320
212	202004	305	2	5	5	1	1200
213	202004	305	1	1	4	1	160
214	202005	305	1	1	13	1	160
215	202005	305	2	12	7	1	1600
216	202006	608	2	1	2	1	180
217	202007	608	2	9	9	1	600
218	202008	608	1	4	11	1	540
219	202008	608	2	8	12	1	60
220	202009	911	1	4	9	2	760
221	202010	911	1	5	2	1	125
222	202010	911	2	15	13	2	2160
223	202010	911	2	3	9	1	960
224	202011	911	1	1	10	1	1000
225	201906	608	2	15	11	1	270
226	201805	305	2	3	3	1	2160
227	201806	608	1	4	2	1	360
228	201806	608	1	1	6	1	450
229	201808	608	2	10	1	1	1440
230	201809	911	1	4	8	1	200
231	201810	911	2	1	12	1	45

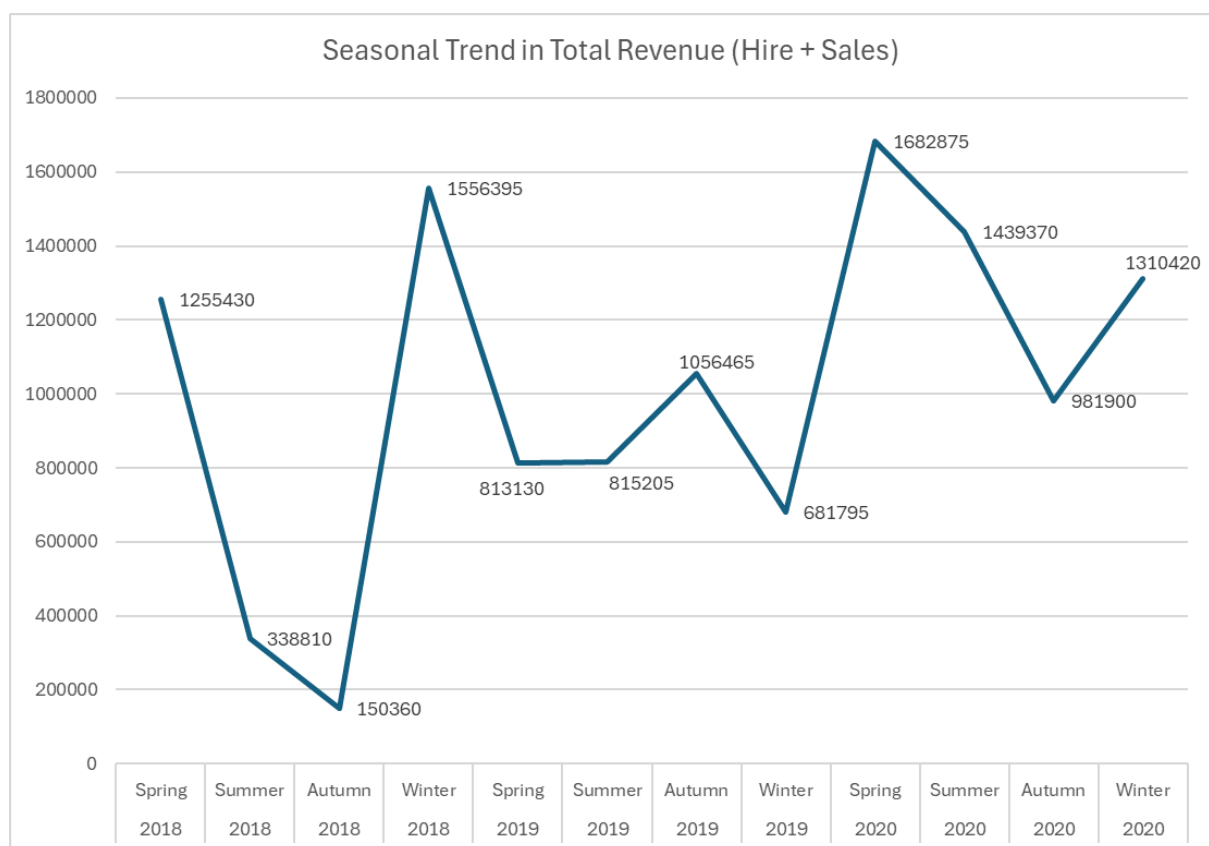
232	201811	911	2	1	11	1	4200
233	201902	1202	2	10	10	1	3000
234	201904	305	1	3	8	1	700
235	201905	305	1	12	8	1	1360
236	201906	608	1	9	14	1	540
237	201907	608	1	7	1	1	1200
238	201909	911	1	12	11	1	300
239	201910	911	1	6	9	1	840
240	201910	911	2	7	11	1	780
241	201911	911	2	1	7	1	2880
242	201911	911	1	10	9	1	120
243	201911	911	2	8	11	1	70
244	201911	911	1	15	11	1	900
245	201912	1202	2	13	13	1	1200
246	202001	1202	2	6	9	1	140
247	202001	1202	2	4	9	1	420
248	202002	1202	2	14	8	1	1600
249	202002	1202	2	10	14	1	600
250	202004	305	2	1	7	1	150
251	202004	305	1	4	6	1	800
252	202004	305	2	13	6	1	1110
253	202005	305	1	4	1	1	1640
254	202005	305	1	4	3	1	1620
255	202005	305	1	13	12	1	1800
256	202007	608	2	15	2	1	160
257	202008	608	1	6	6	1	2400
258	202008	608	2	4	10	1	3840
259	202008	608	2	5	13	1	1200
260	202008	608	1	1	5	1	100
261	202010	911	1	14	5	1	300
262	201810	911	2	1	11	1	160
263	201811	911	2	15	12	1	4000
264	201805	305	2	9	5	1	3960

265	201805	305	2	6	3	1	2400
266	201806	608	1	1	8	1	315
267	201807	608	1	2	7	1	160
268	201807	608	1	4	9	1	80
269	201808	608	2	14	1	1	1200
270	201809	911	1	7	1	1	1200
271	201809	911	1	1	2	1	105
272	201810	911	1	14	9	1	840
273	201811	911	2	11	4	1	4000
274	201812	1202	2	4	6	1	3200
275	201901	1202	2	5	4	1	900
276	201903	305	2	1	9	1	80
277	201903	305	1	14	10	1	480
278	201904	305	2	10	8	1	2250
279	201904	305	2	4	12	1	60
280	201904	305	1	13	5	1	500
281	201904	305	2	4	9	1	420
282	201905	305	1	15	7	1	600
283	201905	305	1	4	5	1	400
284	201912	1202	1	4	2	1	175
285	202005	305	1	12	7	1	70
286	202005	305	2	15	13	1	240
287	202007	608	2	2	6	1	1000
288	202007	608	2	6	10	1	2000
289	202009	911	1	4	1	1	105
290	202009	911	2	13	4	1	1200
291	202010	911	1	7	13	1	640
292	202011	911	1	11	5	1	1000
293	202011	911	2	13	6	1	600
294	202012	1202	2	4	11	1	270
295	201808	608	2	13	1	1	315
296	201903	305	2	10	7	1	2400
297	201905	305	2	2	10	1	2700
298	201908	608	2	10	2	1	180

Descriptive Analysis

Finding 1 - Seasonal Trend in Total Revenue (Hire + Sales)

The findings are based on combined data from 2018 to 2020, capturing all recorded hire and sales transactions across every season. The analysis examines how total revenue fluctuates across Spring, Summer, Autumn, and Winter, revealing differences in seasonal performance that influence MonEquip's overall business activity.



Seasonal Revenue Analysis (2018-2020)

Spring

Revenue: RM 1,255,430 → RM 813,130 → RM 1,682,875
(2018→2020)

Revenue fell in 2019 before rebounding strongly in 2020 to the highest level among all seasons.

Overall: Indicates early-year recovery and strong project initiation after a slowdown in 2019.

Summer

Revenue: RM 338,810 → RM 815,205 → RM 1,439,370 (2018→2020)

Although still lower than Spring and Winter, Summer shows consistent year-on-year growth.

Overall: Demonstrates steady mid-year improvement, suggesting stable project continuation and rising equipment demand.

Autumn

Revenue: RM 150,360 → RM 1,056,465 → RM 981,900 (2018→2020)

Revenue spiked sharply in 2019 but slightly declined in 2020, remaining far above 2018 levels.

Overall: Reflects one-time surges likely due to delayed projects or pre-winter preparations, followed by moderate stability.

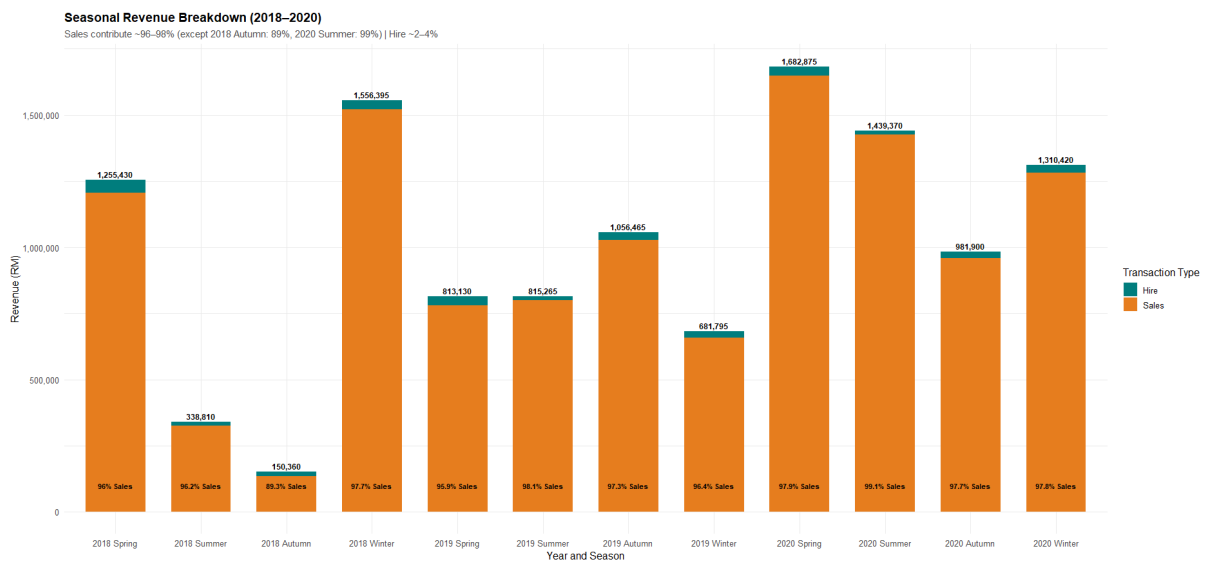
Winter

Revenue: RM 1,556,395 → RM 681,795 → RM 1,310,420 (2018→2020)

Experienced a major decline in 2019 before recovering by **+92%** in 2020, though still below 2018.

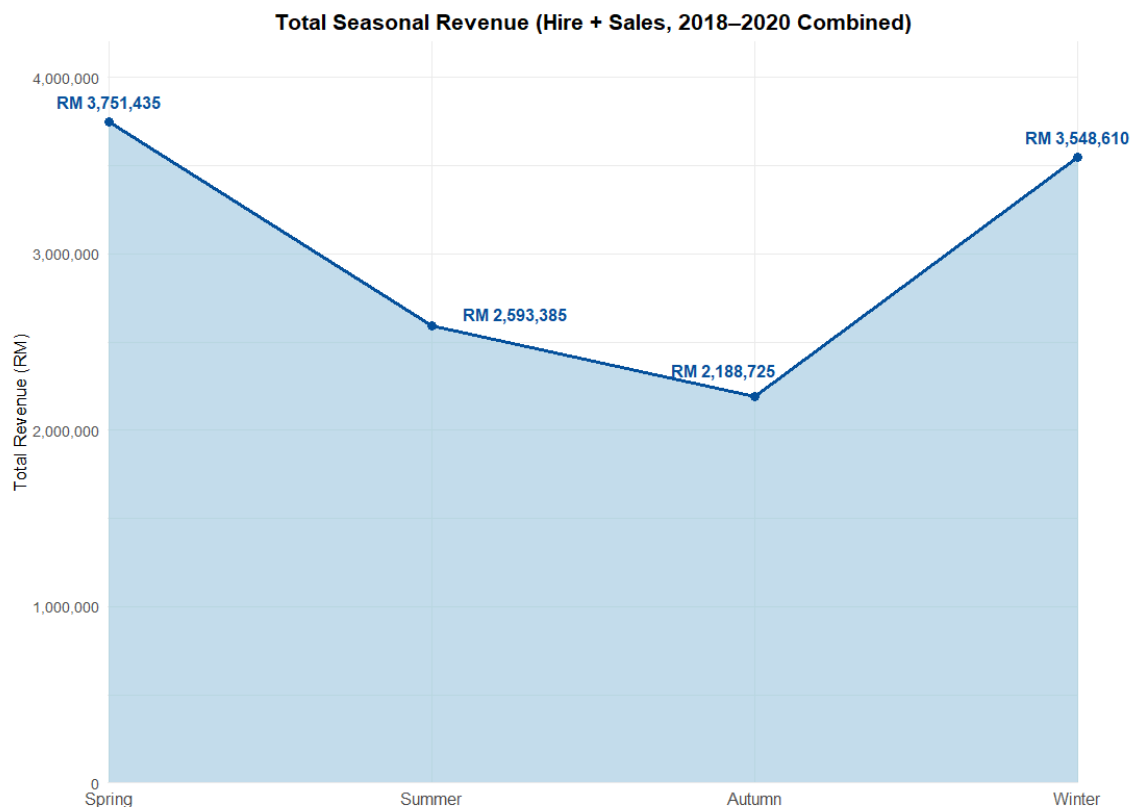
Overall: Suggests cyclical year-end performance tied to project completions and seasonal promotions.

YEAR	SEASONNAME	TOTALHIREREVENUE	TOTALSALESREVENUE	TOTALCOMBINEDREVENUE
2018	Spring	50030	1205400	1255430
2018	Summer	13010	325800	338810
2018	Autumn	16160	134200	150360
2018	Winter	35395	1521000	1556395
2019	Spring	33280	779850	813130
2019	Summer	15605	799600	815205
2019	Autumn	28865	1027600	1056465
2019	Winter	24795	657000	681795
2020	Spring	34775	1648100	1682875
2020	Summer	13570	1425800	1439370
2020	Autumn	22800	959100	981900
2020	Winter	28820	1281600	1310420



Sales vs Hire Contribution

- Sales account for ~96-98% of total revenue; Hire contributes ~2-4%.
- Example: Spring 2020 → Hire RM 34,775 vs Sales RM 1,648,100 (2.1% / 97.9%).
- Confirms that sales performance drives overall revenue patterns.

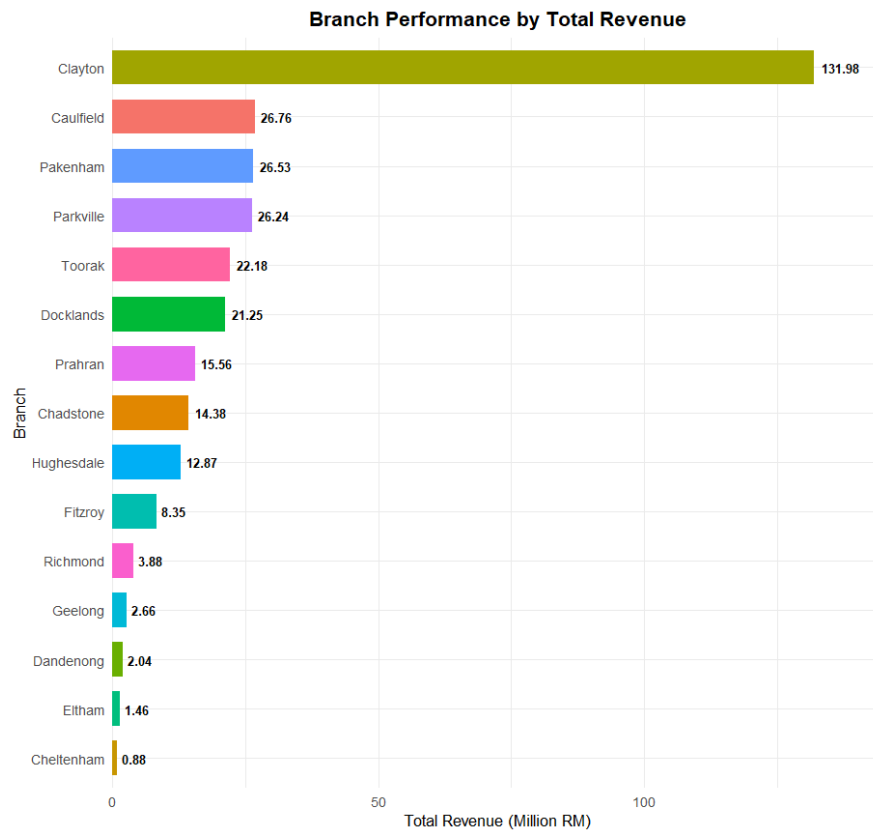


Overall Insights

- Strong in Spring (new projects) and Winter (year-end activities).
- Lower in Summer and Autumn, though both show gradual improvement.

Finding 2 - Branch Performance by Total Revenue (2018-2020)

The findings are based on combined hire and sales revenue data from 2018 to 2020, covering all MonEquip branches across Victoria. This analysis identifies variations in revenue performance between branches.



Branch Revenue Analysis

1. High-Performing Branch

- Clayton achieved the highest total revenue of RM 131.98 million, far surpassing all other branches.
- This confirms Clayton's role as MonEquip's primary operational hub, likely due to its central location, large customer base, and strong sales volume.

Clayton consistently dominates overall branch revenue across all three years.

2. Mid-Tier Branches

- Caulfield (RM 26.76M), Pakenham (RM 26.53M), and Parkville (RM 26.24M) form the mid-tier group.

- These branches exhibit steady and balanced business performance, reflecting solid regional demand and effective sales operations.

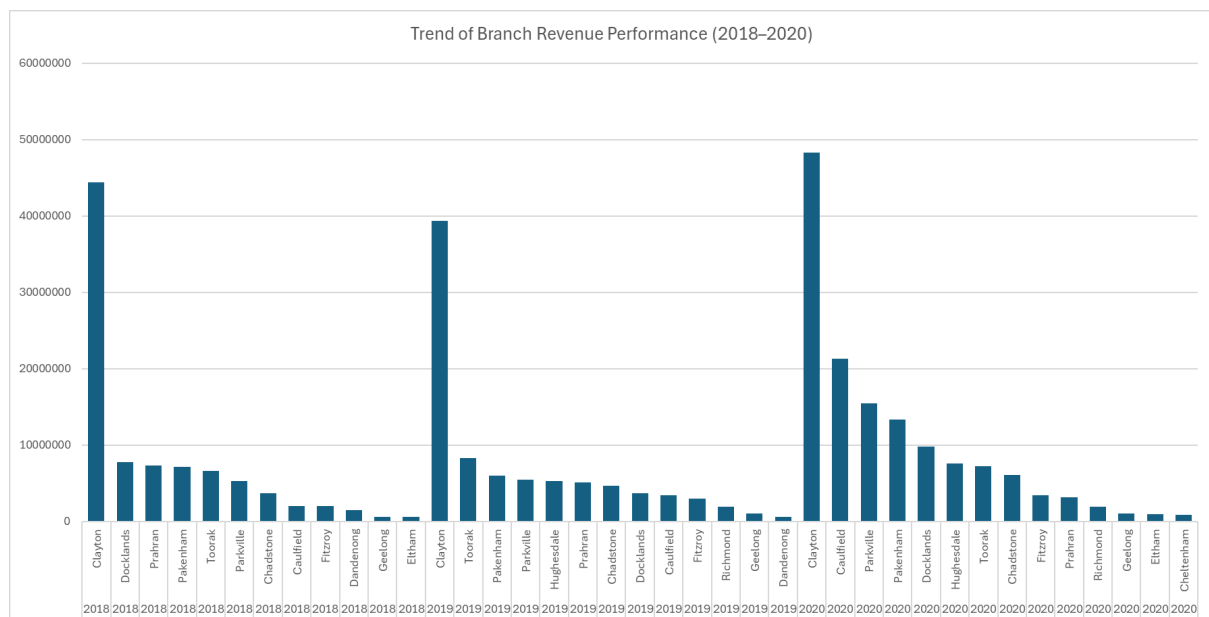
All three branches demonstrated noticeable growth, particularly in 2020, narrowing the gap with the top performer.

3. Moderately Performing Branches

- Toorak (RM 22.18M) and Docklands (RM 21.25M) maintain stable performance levels.

4. Lower-Performing Branches

- Prahran (RM 15.56M), Chadstone (RM 14.38M), and Hughesdale (RM 12.87M) recorded moderate-to-low revenues.
- Richmond, Geelong, Dandenong, Eltham, and Cheltenham - each achieved below RM 10-15M.



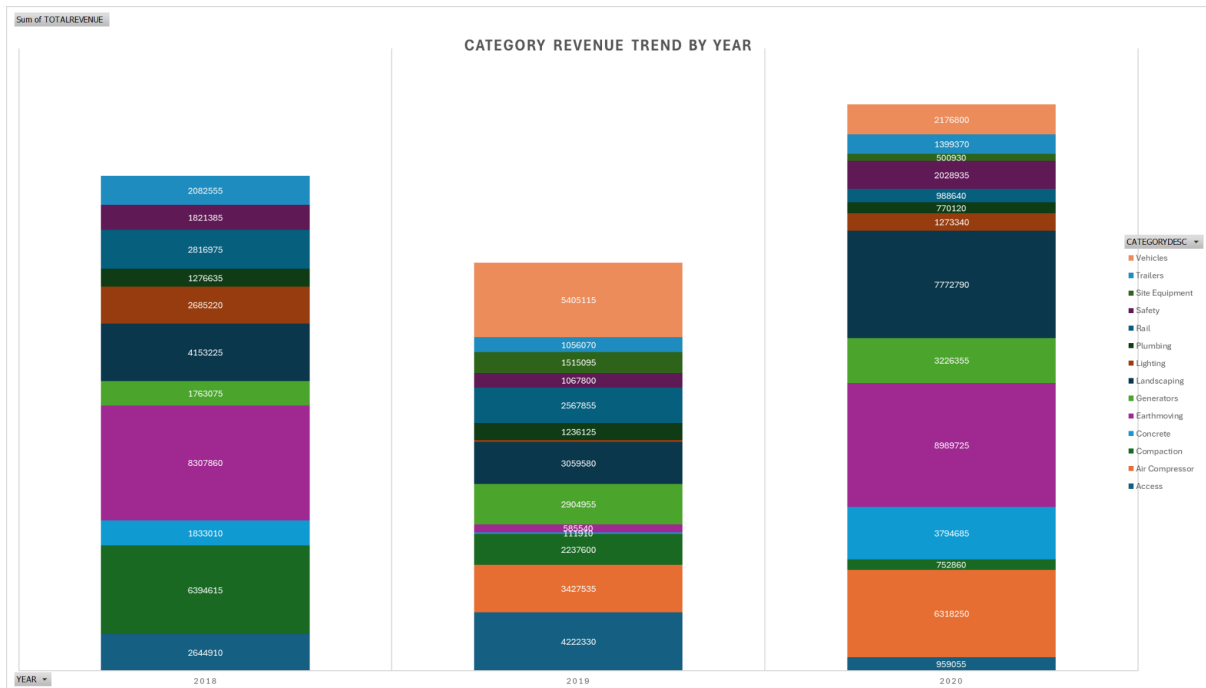
Overall Pattern and Yearly Insight

- Revenue distribution across branches is uneven, dominated by Clayton each year.
- However, regional branches like Caulfield, Parkville, and Pakenham showed strong upward trends in 2020.

BRANCHDESC	YEAR	TOTALSALESREVENUE	TOTALHIREREVENUE	TOTALCOMBINEDREVENUE
Clayton	2018	43728800	619710	44348510
Docklands	2018	7677400	95280	7772680
Prahran	2018	7228200	74115	7302315
Pakenham	2018	7104000	66850	7170850
Toorak	2018	6472200	152160	6624360
Parkville	2018	5238000	56940	5294940
Chadstone	2018	3640000	20180	3660180
Caulfield	2018	1965600	83745	2049345
Fitzroy	2018	1968000	20900	1988900
Dandenong	2018	1468800	15040	1483840
Geelong	2018	567000	11205	578205
Eltham	2018	526400	23800	550200
Clayton	2019	38814950	524370	39339320
Toorak	2019	8174200	152160	8326360
Pakenham	2019	5865600	133700	5999300
Parkville	2019	5428800	56940	5485740
Hughesdale	2019	5167800	142400	5310200
Prahran	2019	5065200	49410	5114610
Chadstone	2019	4603200	40360	4643560
Docklands	2019	3624800	95280	3720080
Caulfield	2019	3326400	111660	3438060
Fitzroy	2019	2952000	20900	2972900
Richmond	2019	1907400	39960	1947360
Geelong	2019	979200	22410	1001610
Dandenong	2019	536400	15040	551440
Clayton	2020	47766100	524370	48290470
Caulfield	2020	21132000	139575	21271575
Parkville	2020	15363000	99645	15462645
Pakenham	2020	13260000	100275	13360275
Docklands	2020	9531200	222320	9753520
Hughesdale	2020	7470400	85440	7555840
Toorak	2020	7176000	50720	7226720
Chadstone	2020	6011600	60540	6072140
Fitzroy	2020	3346500	41800	3388300
Prahran	2020	3045000	98820	3143820
Richmond	2020	1905200	26640	1931840
Geelong	2020	1053000	22410	1075410
Eltham	2020	884800	23800	908600
Cheltenham	2020	852000	26940	878940

Finding 3 - Category Revenue Trend and Transaction Preference (2018-2020)

The combined data from 2018 to 2020 reveals several clear and consistent patterns across MonEquip's product categories.



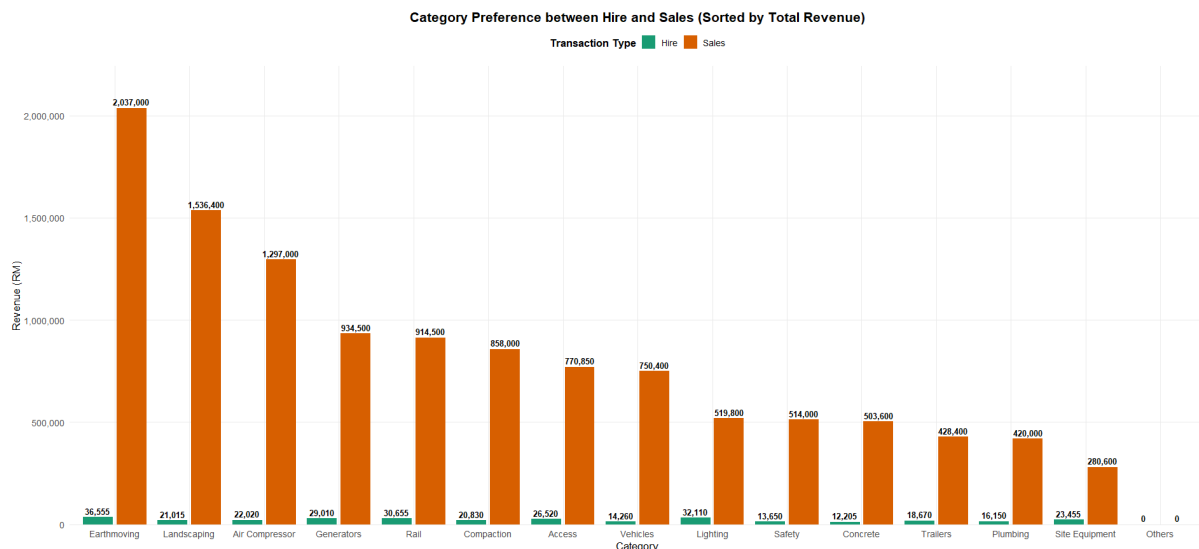
1. Overall Trend (2018-2020)

The stacked bar chart shows that total revenue fell sharply in 2019 but rebounded strongly in 2020, led by high-value categories such as Earthmoving, Landscaping, and Air Compressor.

- **2018:** Top performers were Earthmoving (RM 8.3 M) and Compaction (RM 6.4 M), which together formed the bulk of early revenue and reflected strong project activity.
- **2019:** Revenue declined across almost all categories, especially Concrete (RM 0.11 M) and Earthmoving (RM 0.59 M), suggesting slower construction and equipment spending.
- **2020:** Revenue recovered significantly, with Earthmoving (RM 8.99 M) and Landscaping (RM 7.77 M) regaining leadership, signalling renewed infrastructure investment and increased project demand.

Overall: The sharp drop in 2019 followed by a strong rebound in 2020 shows MonEquip's resilience and growing reliance on heavy-duty equipment rentals for large-scale projects.

2. Category-Level Performance



When combining total figures across 2018-2020, several consistent revenue tiers emerge:

- High-Performing Categories:**
 Earthmoving (RM 2.04 M + RM0.037M), Landscaping (RM 1.54 M + RM0.021M), and Air Compressor (RM 1.30 M + RM0.022M) recorded the highest total sales, reflecting sustained demand for large-scale, high-value equipment used in long-term construction and development projects.
- Moderate-Revenue Categories:**
 Generators (RM 934 K + RM29 K), Rail (RM 914 K + RM31K), and Compaction (RM 858 K + RM21 K) show moderate but stable performance, suggesting periodic demand aligned with specialized or seasonal project use.
- Low-Revenue Categories:**
 Safety (RM 514 K + RM13 K), Concrete (RM 504 K + RM12 K), Trailers (RM 428 K + RM18 K), Plumbing (RM 420 K + RM16 K) and Site Equipment (RM 23 K + RM280 K) contributed the least, implying they are either lower-cost consumables or infrequently

procured assets.

- **The Others category recorded no revenue**, implying the data group is unused during the analysis period.

3. Hire vs Sales Preference

Across all categories, sales transactions overwhelmingly dominate hire transactions:

- Hire revenue typically remained below RM 40 K per category, while sales often exceeded RM 1 M, confirming that ownership is the main customer preference.
- This pattern suggests that MonEquip's clients favour long-term asset ownership rather than short-term rental, aligning with the heavy-machinery nature of its business.

Key Insights

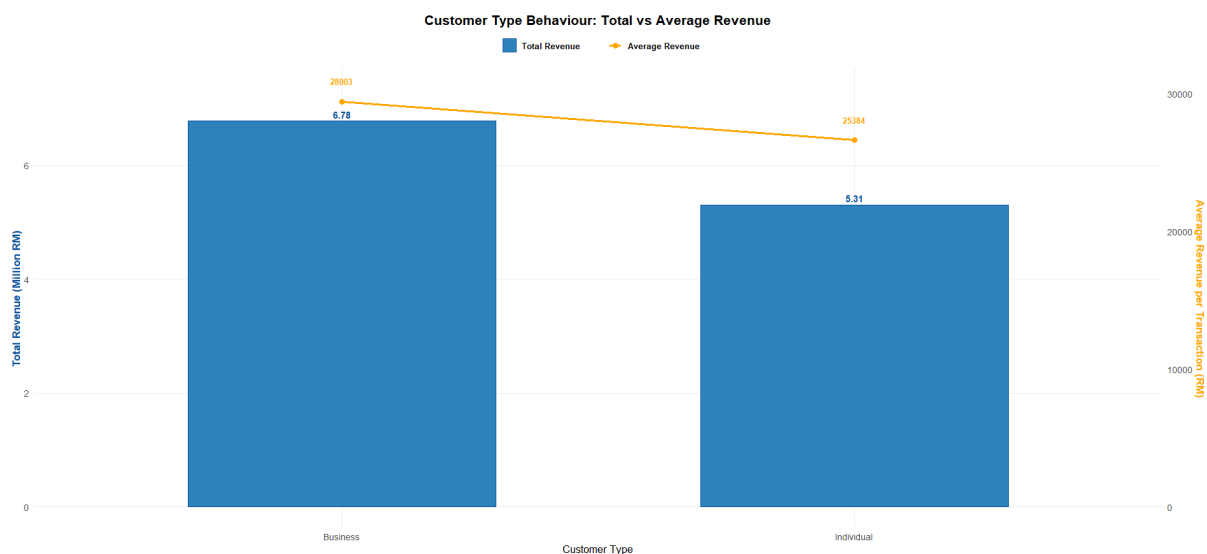
- Sales dominate MonEquip's business model, accounting for over 95% of total equipment revenue between 2018-2020.
- High-value categories such as Earthmoving and Landscaping consistently drive the company's profits and reflect ongoing demand in construction sectors.
- Hire services remain a minor revenue stream, appealing primarily to customers requiring temporary access to specialized tools.

Finding 4 - Customer Type Behaviour (2018-2020)

The findings are based on combined data from 2018 to 2020, capturing all recorded hire and sales transactions for both Business and Individual customer types.

	CUSTOMERTYPEDESC	TOTALHIREREVENUE	TOTALSALESREVENUE	NUMBERTRANSACTIONS	MANUALAVERAGEREVENUE
1	Business	171655	6605150	242	28003.33
2	Individual	145450	5159900	209	25384.45

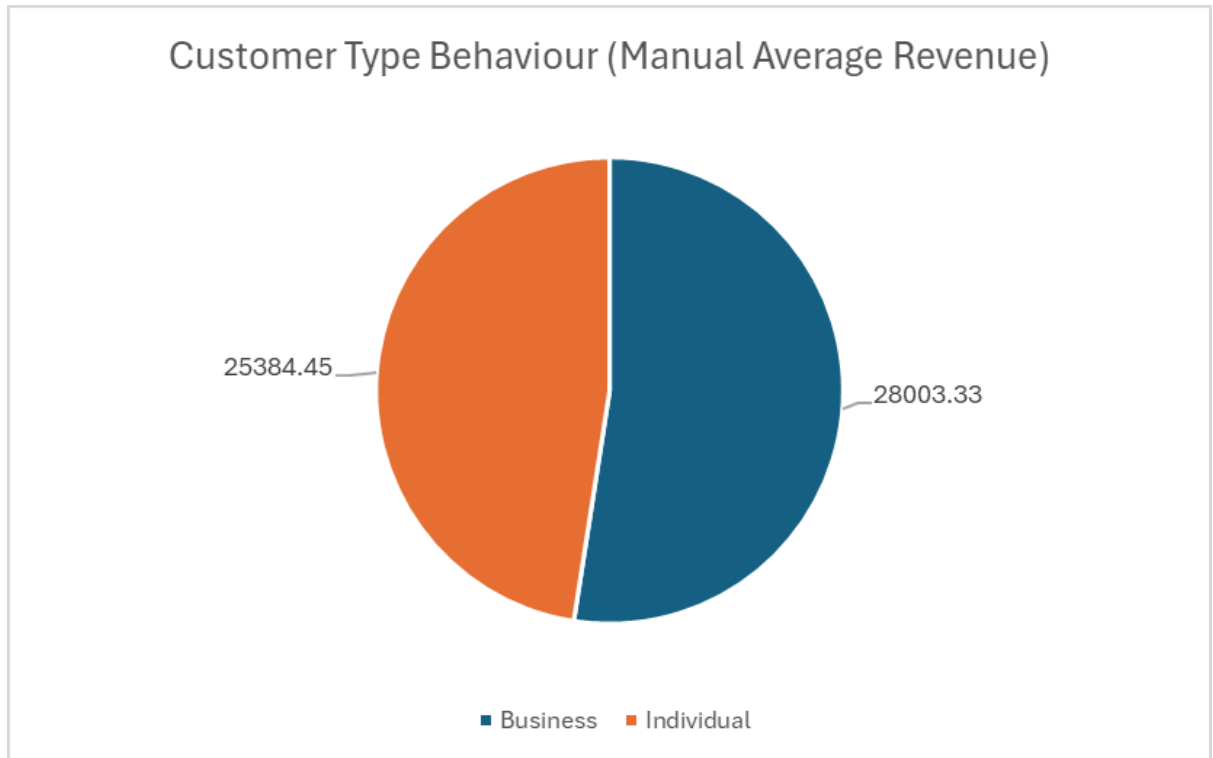
- Business customers generate slightly higher average revenue (RM 28,003) compared to Individual customers (RM 25,384). This suggests that business clients tend to make larger or higher-value purchases and hires, likely due to bulk orders or long-term project requirements.
- Total sales revenue far exceeds hire revenue for both customer types, reinforcing the earlier insight that sales dominate MonEquip's income stream while hire plays a secondary role.
- The average revenue gap between business and individual customers is about RM 2,619 ($\approx 10\%$), which indicates similar spending behaviour, with business clients maintaining a small lead in transaction value.



- The bar-line chart highlights that business customers lead in both total and average revenue, with RM 6.78 million total revenue

versus RM 5.31 million from individuals.

- The accompanying orange trend line (average revenue) confirms the higher per-transaction value among business clients.



- The pie chart illustrates that Business (52.5%) and Individual (47.5%) segments contribute nearly equally to MonEquip's overall revenue potential, signifying a balanced customer base.

From 2018-2020, Business customers are the top revenue contributors due to larger, higher-value transactions, but Individual customers remain a vital and consistent segment that sustains overall sales volume.

Finding 5 - Sales Scale Distribution (High-Value vs Medium-Value, 2018-2020)

The data covering 2018-2020, shows a clear imbalance in sales scale distribution:

	SALESCALENAME	TOTALSALESREVENUE	TOTALTRANSACTIONS
1	High	11728450	145
2	Medium	36600	5

1. Dominance of High-Value Transactions

- High-value sales account for 97% of all sales transactions, generating over RM 11.7 million in total revenue.
- This confirms that MonEquip's income is primarily derived from large-scale or bulk purchases, often associated with corporate clients and infrastructure projects requiring high-end machinery and equipment.

These high transactions form the financial backbone of the company's operations.

2. Limited Medium-Scale Activity

- Medium-scale sales represent only 3% of transactions, indicating that small or moderate purchases are extremely rare.
- The small proportion suggests that customers purchasing at this scale are occasional buyers, possibly for short-term or lower-budget projects.

Such limited activity reflects MonEquip's focus on premium, high-capacity products rather than smaller equipment ranges.

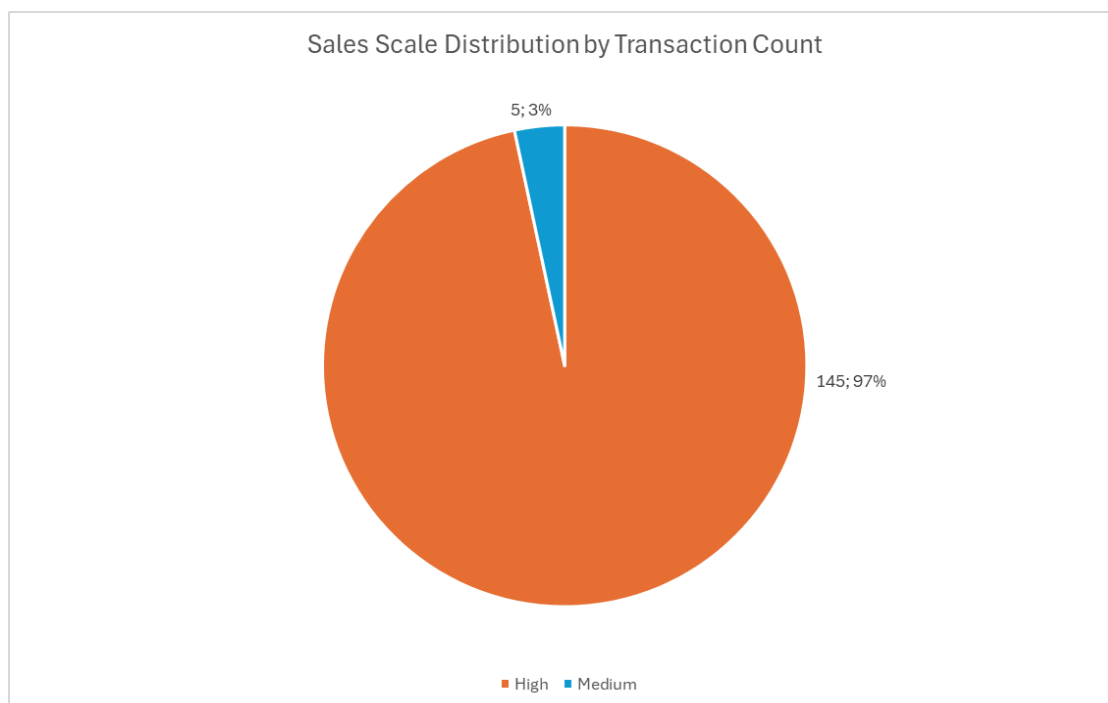
3. Absence of Low-Value Transactions

- The dataset shows no recorded sales below RM 5,000, meaning there are no "low sales scale" transactions within the 2018-2020 period.

- This implies that MonEquip’s pricing structure and target market exclude small-budget customers, focusing instead on enterprise-level clients and long-term investments.

The company positions itself firmly within the high-value industrial market segment.

The pie chart visually reinforces this imbalance, showing that high-scale sales transactions almost completely dominate the company’s revenue profile.



From 2018-2020, MonEquip’s business model relies heavily on high-value sales transactions, with negligible contribution from medium-scale deals.

This pattern suggests that the company’s primary customer are large organisations or contractors, while individual or small-business buyers represent a very minor market share.

Extra Findings:

The unit sales price for each equipment remains the same across all months and years, while the unit hire price may vary depending on the month and year for the same equipment.

```
-- Check if the same EQUIPMENT_ID has different UNIT_HIRE_PRICE across
time

SELECT

    EQUIPMENT_ID,

    LISTAGG(DISTINCT UNIT_HIRE_PRICE, ', ') WITHIN GROUP (ORDER BY
UNIT_HIRE_PRICE) AS PRICE_LIST

FROM

    HIRE

GROUP BY

    EQUIPMENT_ID

HAVING

    COUNT(DISTINCT UNIT_HIRE_PRICE) > 1;
```

All rows fetched: 25 in 0.317 seconds

	EQUIPMENT_ID	PRICE_LIST
1	1	300, 320, 330
2	2	210, 260
3	4	395, 445
4	9	360, 400
5	10	360, 410
6	12	280, 300
7	20	350, 380
8	23	360, 400
9	56	330, 350, 360
10	64	220, 240, 250
11	68	350, 370, 400
12	69	260, 280
13	70	380, 400, 420
14	74	280, 300, 320, 330
15	76	300, 350
16	85	210, 230, 250
17	90	220, 240, 270
18	91	320, 340, 370
19	93	300, 320, 330, 350
20	105	220, 260
21	112	300, 320, 330, 340
22	121	330, 350
23	124	270, 310
24	147	220, 250, 260, 270
25	154	360, 410

Appendix

Data Cleaning Strategies

My strategy is to follow the five rules from the FIT3003 lab materials to check whether the data has any issues, and also to verify the accuracy of the calculations:

1. Duplication problems

My strategy for handling duplication issues is to first identify any records in the table share the same primary key value. When I find duplicates, I immediately remove the incorrect or unnecessary records to ensure all primary keys remain unique. If multiple rows contain similar information, I compare them carefully to determine which one is more accurate and delete the rest. I also cross-check related data in other tables before deciding which record to keep or delete. This approach ensures data consistency and prevents duplicate key errors in the database.

2. Relationship problems or constraint violations

My strategy is to check the data carefully for primary keys, foreign keys, unique values, and check constraints are followed. If there is a problem, I look at the related tables to find the cause and fix the data so that all relationships between tables are correct. This helps keep the database accurate and well-connected.

3. Inconsistent values

My strategy is to make sure all data uses the same format and units. When I find inconsistent values, such as different codes, units, or precision, I compare them with other related tables or the specification. Then, I correct the data so that everything is consistent and makes sense together.

4. Incorrect values

My strategy is to review the data to find any mistakes, such as spelling errors, values outside the normal range, or the wrong data type. When I find an error, I will first refer to the table information and the assignment specification, only consider deleting it or updating it. This keeps my dataset clean, accurate, and ready for analysis.

5. Null value problems

My strategy is to check whether the table has any null values in specific columns. If I find null values, I will decide whether to delete the record or update it with the correct value. Before making any changes, I review other related tables or the specification to make sure the update is accurate and meaningful. This helps keep the dataset complete and reliable for analysis.

Other than that, I also check whether all calculations are correct. If I find any mistakes, I update them to make sure the analysis is accurate and reliable.

Reason Details - record data at the transaction level

From the dataset, it was observed that in the Hire table, several records had NumberHire = 2.

This occurred because two hire transactions were combined into one record when they shared the same EquipmentID, CustomerID, and StaffID.

Such aggregation means the data was not stored at the lowest level of granularity.

	SALES_ID	SALES_DATE	EQUIPMENT_ID	QUANTITY	UNIT_SALES_PRICE	TOTAL_SALES_PRICE	CUSTOMER_ID	↑	STAFF_ID
1	150	15/12/2020 12:00:00	20	4	45500	182000	2	2	37
2	151	15/12/2020 12:00:00	20	4	45500	182000	2	2	37

Similarly, in the Sales table, some customers appeared to have repeated purchases of the same equipment, indicating that the dataset was aggregated above the transaction level.

Adjustment

To improve data accuracy and maintain transaction-level granularity:

1. Include unique identifiers

- Add SalesID and HireID in their respective fact tables to ensure each transaction is uniquely represented.

2. Individual transaction counts

- Define NumberSales = 1 and NumberHire = 1 for every record, preventing the merging of multiple transactions into a single row.

3. Enhance fact table granularity

- Ensure the HireFact table is not designed at a high level of aggregation. Include EquipmentID, CustomerID, and StaffID to capture each transaction separately. This ensures that multiple hires sharing the same customer, equipment, or staff are not combined into a single record (e.g., NumberHire = 2), but are instead stored as distinct transactions.

In the original dataset, several rows in the HireFact table were combined due to this aggregation, resulting in multiple hires being represented as one record.

	HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE_PRICE	TOTAL_HIRE_PRICE	CUSTOMER_ID	STAFF_ID	↑
1	265	04/09/2020 12:00:00	06/09/2020 12:00:00	99	1	20	40	115	3	3
2	273	28/09/2020 12:00:00	01/10/2020 12:00:00	101	3	80	720	131	5	5

	HIRE_ID	START_DATE	END_DATE	EQUIPMENT_ID	QUANTITY	UNIT_HIRE_PRICE	TOTAL_HIRE_PRICE	CUSTOMER_ID	STAFF_ID	↑
1	284	12/10/2020 12:00:00	16/10/2020 12:00:00	151	3	100	1200	134	4	4
2	289	24/10/2020 12:00:00	26/10/2020 12:00:00	143	3	160	960	133	29	29

References

1. What was the total sales revenue in January, 2020?

Ans: 356600

2. How many pieces of equipment were sold in Winter, 2018?

Ans: 19

3. How many equipment was hired by business customers?

Ans: 156

4. What was the total hire revenue in Clayton branch?

Ans: 47670

5. How many trailers were hired by individual customers in Summer?

Ans: 2

6. What is the average sale revenue for Lighting equipment in 2019?

Ans: 28000

7. What is the average hire revenue for Vehicles by individual customers?

Ans: 1866.67

8. How much sales revenue was generated from a high sale in Summer?

Ans: 2551200